

MSc(IT) Semester II [Winter 2025]

IT632: Software Engineering

25-Apr-2025

Duration: 1 hour

Max. Marks: 36

Roll Number _____ Name _____

1. Consider a project time-scale chart with the following tasks: (2 marks)

Task	Duration	Depends on
A	4 days	—
B	3 days	A
C	5 days	A
D	2 days	B, C

Suppose Task C is delayed by **2 days**. Assuming no overlap and that tasks begin immediately after dependencies are met, what is the **minimum additional delay** to the project completion time? Reason your choice with suitable calculations on the time-scale chart.

- (A) 0 days (B) 1 day (C) 2 days (D) 5 days

Ans: (C) 2 days

Explanation: (corresponding to this a time-scale chart should be given)

- **Task A:** Runs from Day 0 to Day 4.
- **Task B:** Starts at Day 4 (after A), ends at Day 7.
- **Task C:** Originally from Day 4 to Day 9; now delayed by 2 days, ends at Day 11.
- **Task D:** Depends on B and C, so it starts after both are completed.
- Originally, Task D could start at Day 9 (after B and C).
- Now, C completes only by Day 11, so D starts at Day 11 and ends at Day 13.

Therefore, the project is delayed by **2 days**.

2. In Earned Value Management (EVM), which of the following formula(s) correctly defines the Cost Performance Index (CPI)? (2 marks)

- (A) $CPI = EV - AC$ (B) $CPI = EV \div AC$ (C) $CPI = AC \div EV$ (D) $CPI = PV \div EV$

What type of conclusions do we make from values of (> 1 , < 1 , $= 1$) for CPI of software projects?

Ans: (B) $CPI = EV \div AC$

Explanation:

The Cost Performance Index (CPI) is a key indicator in EVM that measures the cost efficiency of a project. It is calculated by dividing Earned Value (EV) by Actual Cost (AC).

- If $CPI > 1$, the project is under budget.
- If $CPI < 1$, the project is over budget.
- If $CPI = 1$, the project is on budget.

3. A software system consists of 9 modules. You decide to use incremental integration testing instead of the big bang approach. What is the primary advantage of using incremental integration testing in this context? Reason your choice. (2 marks)

- (A) It eliminates the need for stubs and drivers.
- (B) It allows parallel development and integration of all modules simultaneously.
- (C) It facilitates early detection and isolation of interface defects.
- (D) It reduces the total number of integration test cases to be executed.

Ans: (C) It facilitates early detection and isolation of interface defects.

Explanation:

In incremental integration testing, modules are integrated one at a time (or in logical groups), and each integration is tested immediately. This approach helps in:

Detecting interface errors early, soon after a new module is integrated.

Isolating faults more easily because fewer modules are involved in each integration step.

Managing complexity better than big bang, especially with larger systems (like one with 9 modules).

Option A is incorrect: stubs and drivers are often required in incremental integration to simulate modules that are not yet integrated.

Option B is misleading: parallel development is possible, but incremental testing is inherently sequential in terms of integration.

Option D is incorrect: the total number of tests might actually be higher, but they are more manageable and meaningful.

4. What can be done to reduce coupling in software design? Support your answer with a C++ code snippet. (2 marks)
- (A) Use global variables for data sharing (B) Increase direct dependencies between classes
(C) Apply interfaces and abstraction (D) Make all class methods public

Ans: (C) Apply interfaces and abstraction

A C++ code that uses interfaces and abstraction should be given in support of the choice.

5. In a UML class diagram for a banking system, consider the following classes:

- **Account** (abstract)
- **SavingsAccount** and **CurrentAccount** (both inherit from Account)
- **Customer** (associated with multiple accounts)
- **Transaction** (associated with a specific account) (2 marks)

Which of the following best represents a correct and semantically rich UML design using appropriate relationships? Giving reasons for your choice, construct corresponding UML class diagram.

(A) **Account** is associated with **Customer** using a one-to-one association, **Transaction** inherits from **Account**, **SavingsAccount** and **CurrentAccount** are implemented as interfaces.

(B) **Account** is an abstract class, **SavingsAccount** and **CurrentAccount** use generalization from **Account**, **Customer** is associated with **Account** using a one-to-many association, **Transaction** is associated with **Account** using a one-to-one association.

(C) **Account** is a concrete class, **Customer** inherits from **Account**, **Transaction** is a subclass of **Account**.

(D) **Account**, **Transaction**, and **Customer** all use composition relationships between each other.

Ans: (B)

- Abstract **Account** class allows polymorphism.
- Proper generalization for **SavingsAccount** and **CurrentAccount**.
- One-to-many between **Customer** and **Account** reflects real-world use.
- One-to-one between **Transaction** and **Account** ensures each transaction targets a specific account.

Explanation:**1. Abstract Account Class:**

- ‘Account’ is declared abstract to capture shared behavior.
- Prevents instantiation of generic accounts.

2. Generalization:

- ‘SavingsAccount’ and ‘CurrentAccount’ inherit from ‘Account’ using generalization.
- Supports polymorphism and reuse of shared behavior.

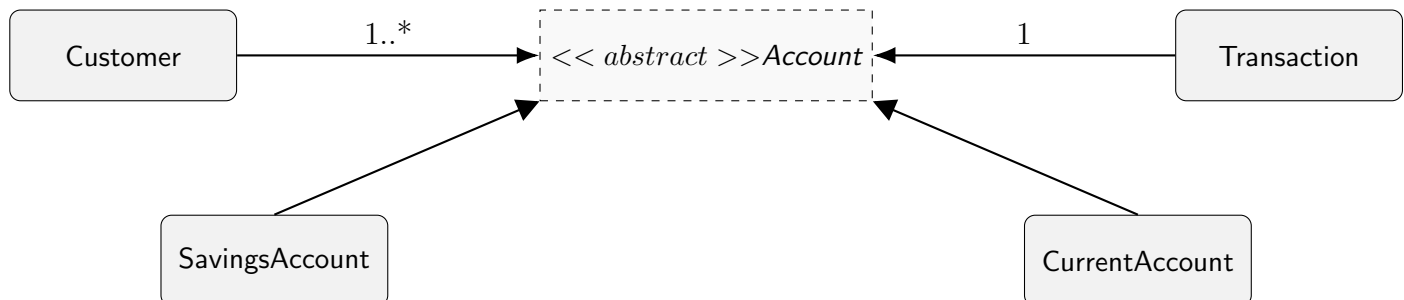
3. Customer-Account Relationship:

- Realistically, a customer can hold multiple accounts.
- Modeled as a **one-to-many** association from ‘Customer’ to ‘Account’.

4. Transaction-Account Relationship:

- Each ‘Transaction’ is associated with exactly one ‘Account’.
- Captures the fact that every transaction targets a specific account.

Option (B) accurately reflects object-oriented and domain-specific semantics. The use of proper abstraction, inheritance, and association makes it both logically correct and extendable for real-world systems.



6. In a UML Use Case diagram for an **Online Shopping System**, following use cases are defined:

- Place Order
- Authenticate User
- Make Payment
- Apply Discount

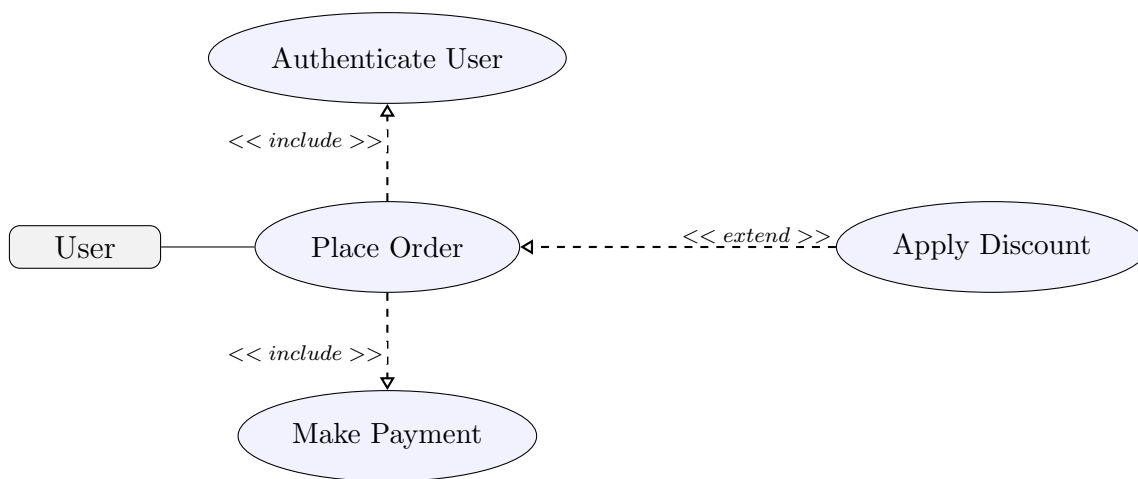
• Track Order

(2 marks)

Which of the following represents the most semantically correct use of `<<include>>` and `<<extend>>` relationships? Giving reasons for your choice, construct corresponding UML Use Case diagram.

- (A) Place Order `<<include>>` Authenticate User, Place Order `<<extend>>` Apply Discount, Track Order `<<include>>` Make Payment
- (B) Authenticate User `<<extend>>` Place Order, Make Payment `<<include>>` Track Order, Place Order `<<extend>>` Apply Discount
- (C) Place Order `<<include>>` Make Payment, Place Order `<<include>>` Authenticate User, Place Order `<<extend>>` Apply Discount
- (D) Track Order `<<extend>>` Make Payment, Apply Discount `<<include>>` Place Order, Authenticate User `<<include>>` Make Payment

Ans: (C) Use Case Diagram given below.



Explanation:

- `<<include>>` is used to model required functionality:
 - ‘Place Order `<<include>>` Authenticate User’ — Every order must be placed by an authenticated user.
 - ‘Place Order `<<include>>` Make Payment’ — Payment is essential to order completion.
- `<<extend>>` is used for optional or conditional behavior:

- ‘Place Order << extend >> Apply Discount’ — Discounts may or may not apply, based on user eligibility or promotions.

3. The relationships are used in the correct direction:

- ‘<< include >>’ arrows point from base use case to reused functionality.
- ‘<< extend >>’ arrows point from optional use case to base.

7. For a Library Management System, that manages books, members, and loans in a small library, following specifications are available. (3 marks)

A Library has multiple Books & Members. Each Book has: title, author, ISBN, & isAvailable (Boolean). Each Member has: memberId, name, & maxBooksAllowed (Integer). A Loan is created when a Member borrows a Book. Each Loan records: issueDate, dueDate, & the returnDate (optional). A Book can only be loaned if it is available. A Member can have multiple Loans, but cannot borrow more than maxBooksAllowed at a time. Write OCL constraints to enforce the following rules:

Rule (a) A Member cannot borrow more books than their maxBooksAllowed.

Rule (b) If a Loan has a returnDate, then the associated book must be available.

Ans: The following OCL invariants enforce the business rules specified for the library system.

Rule (a) A Member cannot borrow more books than their maxBooksAllowed

Explanation: Counts only active loans & ensures they don't exceed borrowing limit.

```
context Member
inv MaxBorrowLimit:
  self.loans->select(1 | 1.returnDate.ocIsUndefined())->size() <= self.
  maxBooksAllowed
```

Rule (b) If a Loan has a returnDate, then the associated book must be available

Explanation: Once returned, the book becomes available to other members.

```
context Loan
inv BookMarkedAvailableOnReturn:
  not self.returnDate.ocIsUndefined() implies self.book.isAvailable = true
```

8. In an **Online Food Ordering System (OFOS)**, a user can browse restaurants, view their menus, and place food orders online. Users must register and log in before placing an order. After selecting

items and confirming the order, they can choose to pay online or opt for cash on delivery. The system sends an order confirmation and estimated delivery time to the user. Restaurant managers can log in to update menu items, accept or reject incoming orders, and view customer feedback. Delivery personnel can log in to see assigned orders and update delivery status. Write textual use case representations (in template format) for the **Place Order** use case. (3 marks)

Ans:**Use Case Name:** Place Order**Primary Actor:** Registered User**Goal in Context:** To order food items from a selected restaurant.**Preconditions:**

- User must be registered and logged in.
- Menu of the selected restaurant is available.

Postconditions:

- Order is recorded in the system.
- Confirmation message with estimated delivery time is sent.

Main Success Scenario:

1. User logs into the system.
2. User browses restaurants and selects one.
3. System displays the menu of the selected restaurant.
4. User adds food items to the cart.
5. User reviews and confirms the order.
6. User selects a payment option (Online/Cash on Delivery).
7. System processes the order.
8. System sends confirmation and estimated delivery time.

Extensions (Alternative Flows):

- (5a.) If items become unavailable during checkout, system notifies user to modify the cart.
(6a.) If payment fails, system prompts user to retry or choose cash on delivery.

9. In UML sequence diagram for an **OFOS** (as in Q.8), following interactions occur:

1. A **Customer** sends a request to **OrderService** to place an order.
2. **OrderService** creates a new **PaymentProcessor** object.
3. The **PaymentProcessor** asynchronously sends a message to **BankGateway**.
4. If payment is successful, **OrderService** sends a confirmation back to **Customer**.
5. If payment fails, an error is returned and the created **PaymentProcessor** is destroyed.

Which of the following options correctly models the above scenario? Reason your choice. (2 marks)

(A) All objects are anonymous. All messages are synchronous. Creation and destruction are not explicitly modeled. No combined fragments used.

(B) Named objects used. Creation shown via `<<create>>`. Asynchronous message from **PaymentProcessor** to **BankGateway**. Conditional logic shown with `alt`. Destruction of **PaymentProcessor** shown with `<<destroy>>`.

(C) Objects use names and types, but interaction starts incorrectly from **BankGateway**. No destruction shown. No combined fragments used.

(D) All messages are asynchronous. Combined fragment is `loop`. Creation and destruction are omitted. Roles not specified.

Ans: (B)

- **Named objects** (e.g., `pp:PaymentProcessor`) are used for clarity.
- `<< create >>` and `<< destroy >>` model object lifecycle explicitly.
- The asynchronous message to **BankGateway** reflects real-world external service behavior.
- The **alt** combined fragment handles success vs failure.

10. A software development team is using Agile practices. For planning a sprint, they have broken down the work into specific tasks with known dependencies and estimated durations in **ideal working days**. (4 marks)

Activity	Description	Duration (days)	Predecessors
A	Define User Stories	2	—
B	Create UI Wireframes	3	A
C	Setup Development Environment	1	A
D	Develop Frontend Components	4	B
E	Develop Backend APIs	5	C
F	Integrate Frontend with Backend	2	D, E
G	Write Unit Tests	2	D, E
H	Perform Sprint Review	1	F, G

Complete the following table after doing necessary calculations.

Ans: Forward Pass (ES/EF), Backward Pass (LF/LS), Slack = LS - ES

Activity	ES	EF	LF	LS	Slack (days)
A	0	2	2	0	0
B	2	5	5	2	0
C	2	3	4	3	1
D	5	9	9	5	0
E	3	8	9	4	1
F	9	11	11	9	0
G	9	11	11	9	0
H	11	12	12	11	0
Critical Path	A → B → D → F → H (total duration = 12 days).				

A → B → D → F → H is the critical path (total duration = **12 days**).

Above critical activities have zero slack & must be completed on time to avoid delaying the project.

11. Give UML activity diagram showing computation of different parameters using CoCoMo. Ensure all modes and variants along with its relevant formulations are depicted in your activity diagram. (4 marks)

Ans: Key Elements in Constructive Cost Model (CoCoMo) Activity Diagram:

The basic model uses the following formulas:

$$\text{Effort (person-months)} = a \times (\text{KLOC})^b$$

$$\text{Development Time (months)} = c \times (\text{Effort})^d$$

Constants a, b, c, d depend on the project mode.

CoCoMo Modes: Organic Mode, Semi-Detached Mode, Embedded Mode

CoCoMo Variants:

1. Basic CoCoMo (Uses only KLOC and mode)
2. Intermediate CoCoMo (Adds 15 cost drivers across product, hardware, personnel, and project)
3. Detailed CoCoMo (Adds phase-wise estimation to the intermediate model.)

Ans:

Constants in Basic CoCoMo

Mode	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semi-Detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Cost Drivers in Intermediate/Advanced CoCoMo

Cost Drivers are attributes that influence the effort required. There are 15 drivers grouped into the following categories:

- **Product:** Required reliability, database size, product complexity.
- **Hardware:** Run-time and memory constraints, platform volatility.
- **Personnel:** Analyst and programmer capability, experience.
- **Project:** Use of tools, schedule constraints, modern practices.

Each cost driver is rated from *Very Low* to *Extra High* and contributes a multiplicative factor (Effort Multiplier) to the effort estimation.

Effort with Cost Drivers in Intermediate CoCoMo:

$$\text{Effort} = a \times (\text{KLOC})^b \times \prod_{i=1}^{15} EM_i$$

12. Use **Equivalence Class Partitioning (ECP)** and **Boundary Value Analysis (BVA)** to design black-box test cases for the function `checkEligibility`. Identify the valid and invalid partitions, and list the boundary test cases. (3 marks)

```
string checkEligibility(int age) {
    if (age >= 18 && age <= 60) {
        return "Eligible";
    } else {
        return "Not Eligible";
    }
}
```

Ans: Equivalence Class Partitioning (ECP):

- Valid class: Age between 18 and 60 (inclusive) $\Rightarrow$ e.g., 30

- Invalid class 1: Age < 18 $\Rightarrow$ e.g., 10
- Invalid class 2: Age > 60 $\Rightarrow$ e.g., 70

Boundary Value Analysis (BVA):

- Lower boundary: 18 $\Rightarrow$ Test with 17, 18, 19
- Upper boundary: 60 $\Rightarrow$ Test with 59, 60, 61

Test Cases:

Test Case ID	Input (age)	Reason / Test Technique	Expected Output
TC1	30	Valid ECP	Eligible
TC2	10	Invalid ECP (<18)	Not Eligible
TC3	70	Invalid ECP (>60)	Not Eligible
TC4	17	BVA (just below lower)	Not Eligible
TC5	18	BVA (lower boundary)	Eligible
TC6	19	BVA (just above lower)	Eligible
TC7	59	BVA (just below upper)	Eligible
TC8	60	BVA (upper boundary)	Eligible
TC9	61	BVA (just above upper)	Not Eligible

13. Analyze the following Java method and compute its **cyclomatic complexity** using Control Flow Graph (both methods). (3 marks)

```

public void process(int num) {
    int i = 0;
    while (i < 5) {
        if (num % 2 == 0) {
            System.out.println("Even");
        } else {
            System.out.println("Odd");
        }

        switch (num) {
            case 1: System.out.println("One"); break;
            case 2: System.out.println("Two"); break;
            default: System.out.println("Other");
        }
        i++;
    }
}

```

Ans: Define Nodes in CFG

We define 12 nodes as:

Node	Statement
1	Method entry and initialization
2	while loop condition
3	if-else condition
4	Print “Even”
5	Print “Odd”
6	switch(num)
7	case 1
8	case 2
9	default
10	i++
11	Loop back to node 2
12	Exit

The control flow edges are:

- $1 \rightarrow 2$
- $2 \rightarrow 3, 12$
- $3 \rightarrow 4, 5$
- $4, 5 \rightarrow 6$
- $6 \rightarrow 7, 8, 9$
- $7, 8, 9 \rightarrow 10$
- $10 \rightarrow 2$

Total edges = 14.

Count Nodes, Edges

- Nodes (N) = 12
- Edges (E) = 14 (determined from control flow)

Cyclomatic Complexity via McCabe’s Formula

$$V(G) = E - N + 2 = 14 - 12 + 2 = \boxed{4}$$

14. Give branch coverage test cases for the following code snippet.

(2 marks)

```
bool authenticate(const std::string& username, const std::string& password) {
    if (username.empty() || password.empty())
        return false;
    if (username == "admin" && password == "admin123")
        return true;
    return false;
}
```

Ans:

Test Case ID	Username	Password	Expected Output	Coverage	Description
TC1	""	"admin123"	false	25%	Username empty
TC2	"admin"	""	false	50%	Password empty
TC3	"admin"	"admin123"	true	75%	Correct credentials
TC4	"user"	"123"	false	100%	Invalid credentials

Total Branch Coverage: 100%